

Report

block 参数

fusion 比之前试出来的 (128, 64, 64) 更好。

(A800)

N=M=K=8192

block_N	block_M	block_K	time
128	128	16	6.9649ms
256	256	16	113.81413500000001ms
256	256	32	59.324331ms
128	64	64	7.612522307692305ms
256	64	64	6.402968999999999ms (fusion best)
128	32	32	10.765322333333335ms
256	32	32	9.410565099999999ms
512	32	32	10.559959333333332ms
64	128	128	9.626644999999998ms

local 复用

那可不可以不复用 local 呢？没试出来，但是

允许两种 fragment 同时存在的，但是应该是不能让它们的生命周期 overlap，但太理解复不复用 local 对生命周期应该本质影响不大，都可以保证一个时刻内需要的寄存器不会吃满，感觉是 tilelang 的

$$a_{i,j}^2 \sum_{j=1}^n a_{i,j}^2$$

程块，它们都会把这些量算一遍，把原本 $O(n^2)$ 级别的计

算 overhead 到了 $O(n^3)$

```
O(n^3)
(tilelang) zwx@allinai2: /mnt/ssd1/zwx-ai/acm-compiler/tilelang$ python kernel/norm_gemm/norm_gemm_fusion.py --block_M 256
Config: M=8192, N=8192, K=8192, block_M=256, block_N=64, block_K=64
dtype=torch.float16
2026-06-22 14:16:25 [TileLang:tilelang.jit.kernel:INFO] (kernel.py:128): TileLang begins to compile kernel `norm_gemm_fusion` with
2026-06-22 14:16:30 [TileLang:tilelang.jit.kernel:INFO] (kernel.py:136): TileLang completes to compile kernel `norm_gemm_fusion`

[PASS] TileLang fused kernel matches reference (rtol=1e-2)
2026-06-22 14:16:31 [TileLang:tilelang.jit.kernel:INFO] (kernel.py:128): TileLang begins to compile kernel `gemm_only` with `out_id
2026-06-22 14:16:37 [TileLang:tilelang.jit.kernel:INFO] (kernel.py:136): TileLang completes to compile kernel `gemm_only`

--- Performance (ms) ---
TileLang Fused: RMSNorm+GEMM 15.4953 ms
TileLang Non-Fused: RMSNorm only 0.3267 ms
TileLang Non-Fused: GEMM only 6.9479 ms
TileLang Non-Fused: sum 7.2746 ms
PyTorch: RMSNorm only 1.1252 ms
PyTorch: GEMM only 5.0153 ms
PyTorch: RMSNorm+GEMM 5.8813 ms
PyTorch: Non-Fus sum 6.1405 ms
Speedup vs TileLang non-fused: 0.47x
Speedup vs PyTorch non-fused: 0.38x

Generated CUDA code saved to: /tmp/norm_gemm_fusion.cu
```

$O(n^2)$

```
(tilelang) zwx@allina12: /mnt/ssd1/zwx-ai/acm-compiler/tilelang$ python kernel/norm_gemm/norm_gemm_fusion.py --block_M 256
Config: M=8192, N=8192, K=8192, block_M=256, block_N=64, block_K=64
dtype=torch.float16
2026-06-22 14:18:27 [TileLang:tilelang.jit.kernel:INFO] (kernel.py:128): TileLang begins to compile kernel `norm_gemm_fusion` with
2026-06-22 14:18:32 [TileLang:tilelang.jit.kernel:INFO] (kernel.py:136): TileLang completes to compile kernel `norm_gemm_fusion`

--- Performance (ms) ---
TileLang Fused:      RMSNorm+GEMM    10.1194 ms
TileLang Non-Fused:  RMSNorm only    0.3261 ms
TileLang Non-Fused:  GEMM only       7.0068 ms
TileLang Non-Fused:  sum              7.3330 ms
PyTorch:             RMSNorm only    1.1066 ms
PyTorch:             GEMM only       4.9923 ms
PyTorch:             RMSNorm+GEMM    5.8984 ms
PyTorch:             Non-Fus sum     6.0989 ms
Speedup vs TileLang non-fused: 0.72x
Speedup vs PyTorch non-fused:  0.58x

Generated CUDA source saved to: /tmp/norm_gemm_fusion.cu
```

把这一块儿注释掉之后效率确实提高了不少..... 但是还是比不上 PyTorch，因为前面说的参数的原因。而且，tilelang 官方实现的 GEMM 甚至比 PyTorch 慢一点。

```
(tilelang) zwx@allina12: /mnt/ssd1/zwx-ai/acm-compiler/tilelang$ python kernel/gemm/bench_gemm.py
Config: M=4096, N=4096, K=4096, block_M=128, block_N=128, block_K=32
dtype=torch.float16
2026-06-22 14:22:27 [TileLang:tilelang.jit.kernel:INFO] (kernel.py:128): TileLang begins to compile kernel `matmul` with dtype=
2026-06-22 14:22:32 [TileLang:tilelang.jit.kernel:INFO] (kernel.py:136): TileLang completes to compile kernel `matmul`

[PASS] TileLang GEMM matches PyTorch (rtol=1e-2)

--- Performance ---
TileLang GEMM: 0.6954 ms (197.63 TFLOPS)
PyTorch GEMM: 0.6217 ms (221.06 TFLOPS)
Speedup vs PT: 0.89x
Gap: 11.9% slower than PyTorch

Generated CUDA source saved to: /tmp/bench_gemm.cu
```

于是，开始思考能不能把 overhead 去掉。发现很困难：

- 如果不用 (block_N, block_M) 的划分方式，用 (block_N, block_K) 来保证每一个 A 里的元素只算一次 elementwise square，那 GEMM 的时候没法写回结果，因为给结果的每一个 block 的贡献可能会被同时到达。
- 如果用 (block_N, block_M) 的划分方式，那就不得不再单独枚举 k 来算 $\sum_{j=1}^n a_{i,j}^2$ ，除非不 fuse。